

LESSON 3

ふだん使わない 機能

目標: SourceTree のどこにその機能があるかを知り、困ったときに思い出せる

ふだん押さない場所に、何があるか

SourceTree の場所	そこから使える機能
ツールバー	スタッシュ、破棄、タグ、ターミナル
左のサイドバー	ブランチ・タグ・リモート・スタッシュ・サブモジュールの一覧
コミット一覧で右クリック	リセット、打ち消し、チェリーピック、リベース、タグ
コミット欄の「コミットオプション」	最後のコミットを上書き（Amend）
ファイルを右クリック	破棄、無視、追跡をやめる、ログを表示、Git LFS
メニュー「リポジトリ」	サブモジュールを追加、Git LFS
メニュー「ツール」→「オプション」	強制プッシュを許可するか、などの設定

POINT

SourceTree のボタンは、裏で **git** のコマンドを代わりに打っているだけ。LESSON 2 の仕組みは、そのまま当てはまります。ボタンがない機能は、このコマの最後にコマンドの一覧で紹介します。

スタッシュ：作りかけを一時的にしまう

こんなとき

ジャンプの調整の途中で、「main のバグを今すぐ直して」と言われた。コミットするには中途半端だが、ブランチは切り替えたい。

SourceTree での操作

- 1 ツールバーの「スタッシュ」を押す
- 2 メッセージ（例：ジャンプ調整の途中）を入れて OK。作業フォルダが、最後のコミットの状態に戻る
- 3 用事が済んだら、サイドバーの「スタッシュ」にある項目を右クリック → 「スタッシュを適用」
- 4 適用後に削除するチェックを入れると、取り出すと同時に一覧から消える

- コミットを作らずに、変更だけを脇に置く
- いくつも積める
- 別のブランチで取り出してもよい
- 新しく作ったファイルは、先にステージしておく確実にしまえる

注意

スタッシュは手元だけのもので、プッシュされない。しまったまま忘れやすいので、長く置くものはブランチを作ってコミットする。

リセットの3つのモードと、打ち消し

戻りたいコミットを右クリック → 「現在のブランチをこのコミットまでリセット」。ブランチの名札をそのコミットへ動かす操作で、どこまで一緒に戻すかを3つのモードから選ぶ。

モード	名札	ステージング	作業ファイル	結果
Soft	戻す	そのまま	そのまま	コミットだけ取り消す。変更はステージ済みで残る
Mixed	戻す	戻す	そのまま	コミットとステージを取り消す。編集は残る
Hard	戻す	戻す	戻す	すべて捨てる。コミット前の編集は戻らない

リセット (reset)

コミットをなかったことにする。履歴から消える。

このコミットを打ち消し (revert)

履歴は消さず、逆の変更をする新しいコミットを足す。同じ右クリックメニューにある。

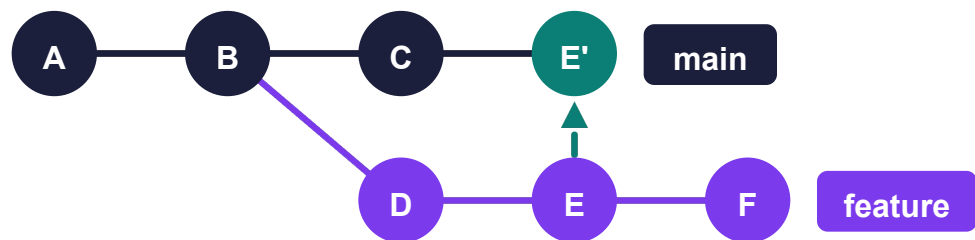
POINT

使い分けはプッシュしたかどうか。手元だけのコミットはリセットで直してよい。プッシュ済みのコミットは、他の人がもう持っているので打ち消しを使う。

cherry-pick：コミットを1つだけ持ってくる

こんなとき

作業中のブランチで、ついでに直したバグ修正（E）だけを、先に main へ入れたい。ブランチ全体はまだ未完成でマージできない。



SourceTree: main をチェックアウト → E を右クリック → 「チェリーピック」

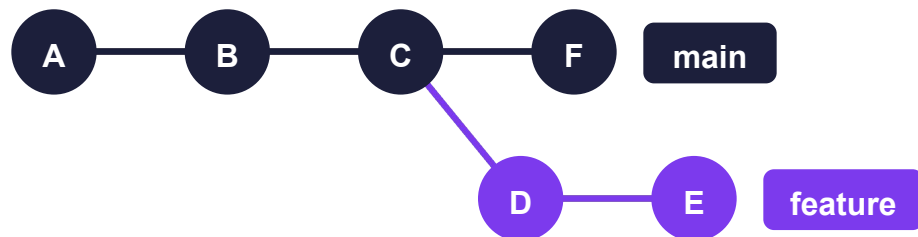
- 指定したコミットの「変更」だけを、今のブランチに当てる
- できるのは別の ID の新しいコミット（E'）
- 間違ったブランチにコミットしたときの移し替えにも使える

注意

E と E' は中身が同じでも別のコミット。使いすぎると、同じ変更が履歴の2か所に入り、あとのマージでコンフリクトの原因になる。

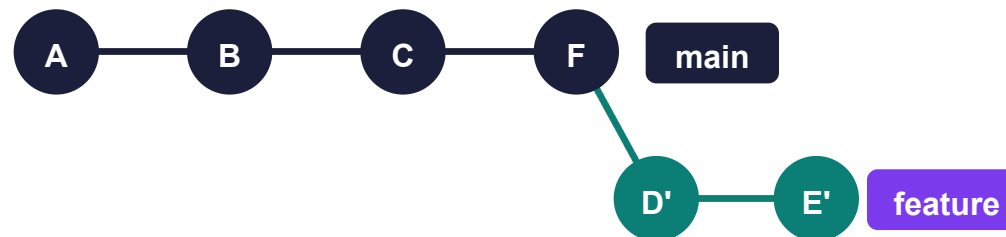
rebase : 枝の付け根を付け替える

rebase の前



feature は C から枝分かれしている。

rebase の後



最新の F から枝分かれしたことになる。D・E は作り直される。

- SourceTree: feature にいる状態で、サイドバーの main を右クリック → 「現在の変更を **main** にリベース」
- マージコミットができず、履歴が一直線になって読みやすい
- merge は「起きたことをそのまま残す」。rebase は「あとから整えて書き直す」

注意

鉄則: **push** 済みで、他の人が使っているコミットは **rebase** しない。相手の手元には古い D・E が残り、同じ変更が二重になって履歴が壊れる。

履歴を整える3つの方法

こんなとき

「ジャンプ追加」「typo 修正」「やっぱり戻す」「もう一回修正」と並んだ履歴を、**main** に入れる前に「ジャンプ追加」の1つにまとめたい。

直前の1つを作り直す

コミットオプション → 「最後のコミットを上書き（**Amend**）」

メッセージの書き間違いや、**.meta** の入れ忘れを直す。足したいファイルをステージしてからコミットする。

直近の数個を編集する

コミットを右クリック → 「...インタラクティブなリベースを行う」

一覧の画面が開き、前のコミットとまとめる（スカッシュ）、メッセージを変える、消す、並べ替える、ができる。

ブランチごと1つにまとめる

GitHub の **Pull Request** → 「**Squash and merge**」

ブランチの全コミットを、1つのコミットにして **main** に取り込む。**SourceTree** ではなく **GitHub** 側のボタン。

注意

左の2つは、コミットを別の **ID** で作り直す操作。リベースと同じく、プッシュする前の自分のコミットにだけ使う。

タグ：動かない名札を付ける

こんなとき

学内発表会に出したビルドは、どのコミットだったか。あとで「あの版」に正確に戻れるようにしたい。

SourceTree での操作

- 1 ツールバーの「タグ」を押す（コミットを右クリック → 「タグ」でもよい）
- 2 タグ名（例：v1.0）とメッセージ（例：学内発表版）を入れる
- 3 「全てのリモートにタグをプッシュ」にチェックを入れて、タグを追加する
- 4 あとでその版を見るには、サイドバーの「タグ」にある v1.0 をダブルクリック

- ブランチはコミットのたびに進む名札
- タグはその場から動かない名札
- 4 の操作をすると、LESSON 2 の detached HEAD になる（確認の画面が出る）

注意

ふつうのプッシュでは、タグは送られない。3 のチェックを忘れると、自分の PC にだけタグが残る。あとから送るときは、プッシュの画面で「すべてのタグをプッシュ」にチェックを入れる。

Git LFS : 大きなファイルを外に置く

Git のリポジトリ

version `https://git-lfs...`

oid `sha256:4d7a2146...`

size `84977953`

3行のテキスト（引換券）だけが入る

ID に対応



LFS 用の保管場所

Boss.psd の本体（約 85 MB）

必要な版だけをダウンロードする

- SourceTree: メニュー「リポジトリ」→「**Git LFS**」で初期化し、対象の拡張子（*.psd など）を登録する。設定は `.gitattributes` に書かれるので、これもコミットする
- clone しても、過去の版の本体は取ってこない。「全員が全履歴を持つ」をバイナリだけやめる仕組み
- ロック: ファイルを右クリック → 「**Git LFS**」からロックする。編集中のファイルを、他の人は読み取り専用にする

注意

全員が **Git LFS** を入れていないと、引換券のテキストが画像のふりをして届き、開けない。設定より前にコミットした分は **LFS** にならない。**GitHub** の **LFS** には容量と転送量の上限がある。

サブモジュール：リポジトリの中のリポジトリ

こんなとき

自作の共通ライブラリ（シングルトンの基底クラスなど）を、複数のゲームで使いたい。コピーして回ると、直すたびに全部を直すことになる。

SourceTree での操作

- 1 メニュー「リポジトリ」→「サブモジュールを追加」
- 2 ライブラリの URL と、置く場所（例：Assets/Lib）を入れる
- 3 サイドバーの「サブモジュール」に表示される。ダブルクリックすると、ライブラリ側のリポジトリが開く
- 4 ライブラリを新しい版にしたら、ゲーム側でもその変更をコミットする

- ゲーム側が記録するのは、「ライブラリのどのコミットか」という ID だけ
- ライブラリ側が進んでも、ゲーム側が ID を更新するまで古い版のまま
- 版を固定できるので、勝手に壊れない

注意

取得のしかたによっては、サブモジュールのフォルダが空になる。GitHub の「Download ZIP」には中身が入らない。「Lib フォルダが空でエラーだらけ」の原因はたいていこれ。

戻せる操作と、戻せない操作

戻せる（コミット済みのもの）

- リセットで戻りすぎたコミット
- 失敗したリベースやマージ
- 消してしまったブランチ
- Amend で上書きする前のコミット

戻せない

- 「破棄」した、コミット前の編集
- **Hard** リセットで消えた、コミット前の編集
- 「削除」した、未追跡のファイル
- 強制プッシュで上書きした、他の人のコミット

危ない操作	SourceTree での安全な使い方
破棄・削除	確認の画面に出るファイル名を読んでから OK する。迷うものは先にスタッシュへ
Hard リセット	ファイルステータスが空（コミット前の編集がない状態）であることを確かめてから
強制プッシュ	最初は使えない設定になっている。オプションで許可するときは「安全な強制プッシュ」も有効にする

POINT

線引きは1つ。**Git** が守ってくれるのは、コミットしたものだけ。危ない操作の前にコミットしておけば、ほとんどの失敗は取り戻せます。

SourceTree にボタンがない機能

ツールバーの「ターミナル」を押すと、そのリポジトリでコマンドを打てる画面が開く。次の機能は、そこから使う。

機能	できること	コマンド
reflog	消してしまったコミットやブランチの ID を探す	git reflog
bisect	バグが入ったコミットを、半分ずつ絞って探す	git bisect start / good / bad / reset
worktree	作業フォルダを増やし、別のブランチを同時に開く	git worktree add ../フォルダ ブランチ名
フック	コミットやプッシュの直前に、検査を自動で動かす	.git/hooks にスクリプトを置く
スパースチェックアウト	指定したフォルダだけを取り出す	git sparse-checkout set フォルダ
部分クローン	ファイルの中身を、必要になったときだけ取る	git clone --filter=blob:none URL
無視の原因調べ	どのルールで無視されているかを表示する	git check-ignore -v ファイル名

POINT

書き方は覚えなくてよい。名前と「何ができるか」だけ知っていれば、必要になったときに調べられます。特に reflog は、失敗したときの最後の頼みです。

Unity では

Unity でコンフリクトを減らす

シーンとプレハブは手ごわい

- テキストだが、中身は ID と数値の長い並び
- 目印が入っても、人が読んで直すのは難しい
- 直し方を間違えると、シーンが開けなくな

ぶつからない分け方

- 同じシーンを、同時に2人で触らない
- 触る前に「今から Stage1 を触る」と声をかける
- 部品はプレハブに分け、シーンには置くだけにする

毎日の習慣

- 作業を始める前に pull する
- 小さくコミットし、こまめに push する
- ブランチを何日も放置しない
- .meta を忘れずに一緒にコミットする

POINT

「声をかけてから触る」は、LESSON 1 のロック方式を人の約束で行っているのと同じ。合体しにくいファイルには、ロックの考え方が今も有効です。

チーム開発の流れ (GitHub Flow)



Pull Request とは

- 「このブランチを **main** に入れてください」という依頼
- 変更の差分を見ながら、コメントをやりとりできる
- GitHub の機能で、Git 本体のコマンドではない

レビューで見ること

- 手元に持ってきて、ゲームが動くか
- 関係のない変更が混ざっていないか
- **.meta** の入れ忘れがないか

ルール: **main** には直接コミットしない。必ずブランチと **Pull Request** を通す

GitHub 側にある機能

機能	できること
ブランチ保護	main への直接 push を禁止し、レビューを通ったものだけマージできるようにする
Issue	やることやバグを、番号付きで管理する。コミットに「Closes #12」と書くと、マージ時に自動で閉じる
Draft Pull Request	作りかけの段階で見せて、早めに意見をもらう。完成するまでマージできない
Actions	push や Pull Request のたびに、ビルドやテストを自動で実行する
Releases	タグにビルドしたファイルと説明を付けて、配布する
Fork	他の人のリポジトリを自分のアカウントに複製し、そこから変更を提案する
Blame・History 表示	行ごと・ファイルごとの履歴を、ブラウザで見る

POINT

これらは **Git** ではなく **GitHub** の機能。別のサービスでは名前や形が違う（GitLab では Pull Request を Merge Request と呼ぶ）。Git 本体の知識は、どのサービスでも共通です。

注意

チーム開発で避けること

main に直接 push する

動かない状態が **main** に入ると、全員の作業が止まる。ブランチと Pull Request を通す。

git push --force を使う

リモートの履歴を自分の手元の内容で上書きする。他の人のコミットが消える。

大きなファイルをそのまま入れる

GitHub は 100 MB を超えるファイルを受け付けない。動画や大きな素材は Git LFS を使うか、別の場所に置く。

パスワードや API キーを入れる

後からファイルを消しても、過去のコミットには残り続ける。入れてしまったら、そのキー自体を作り直す。

POINT

4つとも、原因は「履歴は全員に配られ、消えない」という分散型の性質。便利さの裏側として覚えておく。

困ったときの機能マップ

困りごと	機能	SourceTree での場所
作りかけのまま、別のブランチへ行きたい	スタッシュ	ツールバー「スタッシュ」
直前のコミットを直したい	Amend	コミットオプション
プッシュ前のコミットを取り消したい	リセット	コミットを右クリック
プッシュ済みの変更を取り消したい	打ち消し（revert）	コミットを右クリック
別のブランチの、あのコミットだけ欲しい	チェリーピック	コミットを右クリック
履歴を一直線にしたい・まとめたい	リベース	コミットやブランチを右クリック
提出した版に印を付けたい	タグ	ツールバー「タグ」
大きな素材を扱いたい・ロックしたい	Git LFS	メニュー「リポジトリ」
共通のコードを複数のゲームで使いたい	サブモジュール	メニュー「リポジトリ」
コミットやブランチを消してしまった	reflog	なし（ターミナルでコマンド）
いつから壊れていたのかわからない	bisect	なし（ターミナルでコマンド）
ブランチの切り替えを待ちたくない	worktree	なし（ターミナルでコマンド）